

dealing with data imbalance

how to deal with data imbalance problems? two main strategies...

- **cost-sensitive classifiers:**

during the training phase different weights are given to samples according to their class membership $\Rightarrow$ LARGER WEIGHTS TO ERRORS ASSOCIATED TO THE MINORITY CLASS IN THE LOSS FUNCTION

- **data level preprocessing:**

- 1 undersampling: a subset of samples of the majority class is removed
 - ★ may discard potentially relevant samples
 - 2 oversampling: a subset of **new** minority class samples is added by replicating samples or by "estimating" new ones (SMOTE method)
 - ★ may cause overfitting
 - 3 hybrid: undersampling+oversampling
- ARTIFICIAL MINORITY CLASS SAMPLES "SIMILAR" TO THE REAL ONES

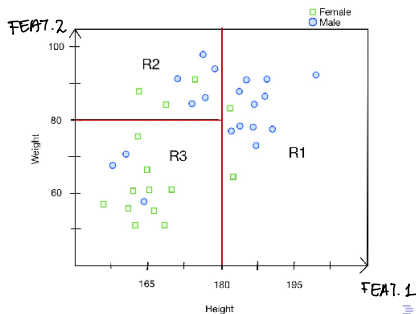
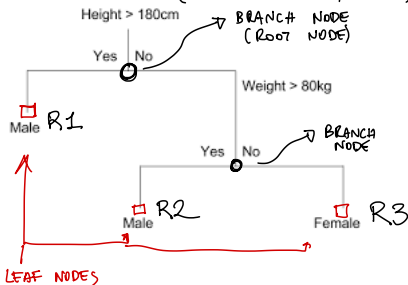
imbalanced-learn package

decision trees

machine learning model for **classification** (binary and multiclass) and regression

- easily interpretable models and representable with a tree structure (widely used in medical applications)
- idea: the tree represents a sequence of **splitting** rules operated on the features' domain
- **branch** nodes (splitting decisions) and **leaf** nodes (final target assignment)
- the class of leaf nodes is the one of the largest number of training observations falling into the node

2-DIMENSIONAL DATASET (2 FEATURES: HEIGHT, WEIGHT)
(BINARY CLASS: MALE, FEMALE)



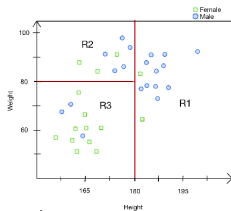
- multiclass classification context

$$TR = \{(x^i, y^i) : x^i \in \mathbb{R}^n, y^i \in \{1, 2, \dots, K\}, i = 1, \dots, m\}$$

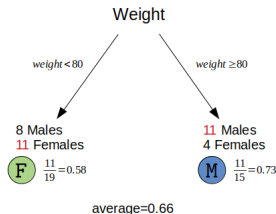
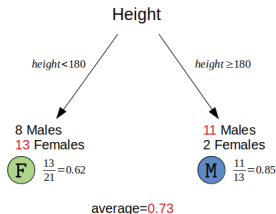
- given feature x_j and threshold s , the splitting regions $R_1(j, s), R_2(j, s)$ are
 $\downarrow$ FOR CONTINUOUS OR DISCRETE/NOT BINARY FEATURES
 $R_1(j, s) = \{x : x_j < s\}$ and $R_2(j, s) = \{x : x_j \geq s\}$

no need of threshold for binary features!

- the class of a region is its majority class (ALSO FOR BRANCH NODES)
- the "quality" of a splitting is the ability to separate observations of different classes according the **purity** ratio $\frac{\# \text{ majority class}}{\# \text{ observations in the region}}$



↑ PURITY $\Rightarrow$ BETTER SPLIT

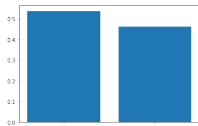


often entropy is used as **impurity** measure (instead of the purity ratio)

$$H = - \sum_{k=1, K} f_k \log(f_k)$$

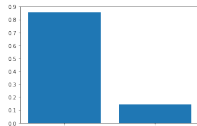
where f_k is the frequency of occurrences of class k in the region

bad splitting



↑ ENTROPY
↓ SPREAD

good splitting



↓ ENTROPY
↑ SPREAD

how to train a **classification tree**?

in principle determine regions and thresholds $R_\ell(j, s)$ with $j = 1, \dots, n, \ell = 1, \dots, M$ minimizing the overall entropy of all splittings

DEPTH OF THE
TREE = NUMBER
OF LAYERS

too expensive to consider every partition of the feature space in M regions!

⇒ a top-down **greedy** approach: **recursive binary splitting**

↘ NP-HARD

- 1 at each step determine feature x_j and threshold s such that the splitting

$$R_\ell(j, s) = \{x : x_j < s\} \text{ and } R_{\ell+1}(j, s) = \{x : x_j \geq s\}$$

minimize the average entropy of the two formed regions among all possible features and thresholds

- 2 stop if some condition is reached (e.g. number of points in a region, number of nodes, entropy of the splittings)



recursive binary splitting

- greedy $\Rightarrow$ at each step the best current split is added not considering the split leading to the best final tree
- computing the best current split (x_j and s) can be done efficiently
- can be used for mixed features (continuous, categorical,...)

example: dataset of 303 patients with binary target indicating the presence of heart disease (the specified value in the branch node corresponds to left branch)

THALASSUM TEST:
{a, b, c, d}

C_a : CONTINUOUS
FEATURES

• THE SAME FEATURE
USED AT DIFFERENT
NODES

• DIFFERENT LENGTH PATHS

• ANOMALOUS
USELESS SPLITS

